

CacheShift

A money-saving gateway for AI apps that checks whether the savings are quietly hurting answer quality

Project proposal, draft v3 · Spencer Aldrich and Anthony Capraru (two-person team) · EC601 · Area 4: Cloud, Systems & Distributed Computing · 22 September 2026

"CacheShift" is a working name. References are listed in full in §13. Status labels used below:

Verified = checked against the source; **Not run** = the test is planned but not done yet; **Checked on paper** = confirmed from documentation, not yet tried ourselves.

Key terms

Term	What it means in this proposal
LLM (large language model)	An AI model that reads and writes text, such as GPT-4 or Llama.
Token	A small piece of a word. AI providers charge per token read and written.
Strong model / weak model	An expensive, more capable model vs. a cheap, less capable one.
Router	A program that decides, for each question, whether to send it to the strong or the weak model.
Cutoff (α)	The router scores each question; questions scoring at or above α go to the strong model.
Percentile	The score below which a given share of questions falls. The 50th percentile is the middle score.
Semantic cache	A store of past questions and answers. If a new question <i>means</i> the same as a stored one, the cache returns the stored answer instead of calling a model.
Embedding	A list of numbers that represents a question's meaning. Similar meanings give similar numbers.
Similarity threshold	How close two questions' embeddings must be before the cache treats them as the same question.
Cache hit / miss	A hit: the cache answers the question. A miss: the question goes on to the router.
False hit	The cache reuses an answer for a question that only <i>looked</i> similar, so the answer is wrong.
Inherited error	The cache reuses an answer that was wrong from the start, because the model that first wrote it got it wrong.
Drift	How far the router's real share of strong-model calls (and its cost and quality) moves from the share it was set up for.
Gateway	A small server that sits between an app and the AI models. The app sends questions to the gateway, and the gateway runs the cache and the router.
Replay mode	Running the gateway with answers taken from a recorded dataset instead of live, paid AI calls. It makes experiments cheap and repeatable.
Benchmark / correct answer	A standard set of test questions with correct answers written by people, such as MMLU (general knowledge) or GSM8K (math).
Rewording	Asking the same question in different words. We use groups of rewordings to test the cache.
Confidence interval (margin of error)	A range that shows how certain a result is. We use 95% intervals from <i>bootstrapping</i> : re-sampling the questions many times and seeing how much the result moves.

Term	What it means in this proposal
Quantization	Storing a model's numbers with fewer bits (for example 4 instead of 16), so it runs cheaper but may lose some accuracy.

1. The proposal in one paragraph (user, decision, cost of mistakes)

Our user is an engineer who runs an app feature built on an LLM, under a monthly budget. To save money, many such engineers put two tools in front of their models: a **semantic cache** and a **router**. Open-source tools like vLLM Semantic Router, and commercial products like LiteLLM and Kong, now offer both. The engineer decides what share of questions the router sends to the strong model, how strict the cache should be, and whether the budget they planned still holds once the cache is turned on. Mistakes in one direction are **silent**: the cache repeats wrong answers, or the router sends fewer questions to the strong model than planned, and answer quality drops without any dashboard noticing. Mistakes in the other direction are **visible and cheap to fix**: more questions go to the strong model, and the bill shows it. Because the costly mistake is the invisible one, CacheShift must report **how good the cache's answers are and how far the router has drifted, each with a margin of error, shown next to cost, never blended into one score.**

2. Who it is for: the six questions from the Phase 1 brief

Question	Answer
Who is the user?	An engineer on a small team running an LLM feature (a support chatbot, a coding helper, a document-labeling pipeline) under a budget someone else set, with a cache and a router in front of their models.
What decision do they make with our output?	How much traffic to send to the strong model; how strict to make the cache; whether to re-tune the router after turning the cache on.
What does a wrong answer cost, in each direction?	Toward saving money: a silent drop in quality. Toward spending money: a visible overspend, fixed as soon as the bill arrives.
What limits their setting?	They pay per token; nobody on the team specializes in AI systems; their dashboards show how often the cache is used and how much it saves, but not whether its answers are right [1].
Who else is affected?	The budget owner, who sees spending; the app's users, who see answers but never the cache; a sustainability lead, if there is one, who cares about energy and carbon.
What exists today, and why isn't it enough?	Gateway products combine caches and routers, and research routers are tuned to a budget on a fixed set of sample questions [2, 3]. None of them report how the cache shifts the router or how good the cached answers are. Zhu et al. study the combination but assume a perfect cache [4].

3. The problem

AI costs keep growing, and teams stack tools to cut them: caches [5, 6], routers [2, 7, 8], compressed self-hosted models [9, 10, 11], and routers that choose models by carbon [12]. Each tool has been studied alone. When stacked, they affect each other in two main ways.

1. The cache makes the router drift from its budget. A router like RouteLLM decides one question at a time: score at or above α goes to the strong model. But α is chosen from a fixed set of sample questions, as the percentile that sends a planned share (say 50%) to the strong model [2, p.

3; 3]. A cache in front answers some questions before they reach the router. The router now sees a different mix, so the same α sends a *different* share to the strong model. No single routing decision is wrong, but the budget is no longer what was planned. RouteLLM's own documentation warns that "the % of calls routed to each model will differ based on the actual queries received" [3]. **Verified** from RouteLLM's code.

A simple example (made-up numbers): the router is set so 500 of 1,000 questions go to the strong model. A cache then answers 300 easy questions. The router now sees 700 questions, and the same 500 still score above α , so **71% of what the router sees goes to the strong model, not 50%**. If the cache instead absorbed mostly hard questions, the share would fall below 50%, and quality would quietly drop. Nobody has measured which way real caches push.

2. The cache repeats mistakes. A cached answer is whatever was stored, often a weak model's answer. The cache then gives that answer to everyone who asks a reworded version of the question. False hits add a second way to be wrong.

A third, smaller issue is energy. Routers that choose models by carbon use one fixed energy number per model, measured with one request at a time [12, p. 6]. But real energy per token changes about 28–32 times depending on how busy the computer is [13, p. 5]. This is our stretch goal.

Funding agenda. NSF's Design for Environmental Sustainability in Computing program (NSF 23-532) funds work on "the substantial environmental impacts that computing has through its entire lifecycle" [14]. **Industry signals.** In August 2026, the vLLM Semantic Router project opened a work item requiring that "Evaluation reports quality and false-hit risk together with hit rate, latency, and avoided inference cost" [1]. Y Combinator's Fall 2026 Requests for Startups describes data centers as "running out of electricity and land" [15].

4. What already exists, and what is still open

Work	What it does	What it leaves open
Zhu et al., NeurIPS 2023 [4]	Tunes a cache and a model selector together	Assumes a perfect cache (a "semantic search oracle" [4, p. 2]), so no false hits and no effect on what the selector sees
LLMBridge [16]	A working gateway with a cache and model selection	Tests the parts separately; doesn't measure how they affect each other
RouteLLM [2], RouterBench [8], FrugalGPT [7]	Routers, and ways to score them	No cache in front; tuned on a fixed set of sample questions [3]
GPTCache [5], vCache [6]	Semantic caches; vCache keeps the cache's error rate under a limit	No router behind the cache
GAR [12]	Chooses models by carbon, with accuracy and speed limits	Leaves out dollar cost [12, p. 9]; one fixed energy number per model [12, p. 6]
RequestRouter [11]	Chooses a compressed model version per request, with measured energy and quality	One self-hosted model; no cache, no API models, no dollars [11, p. 2]

What we claim, stated narrowly. No published work measures (a) how far a real, imperfect cache makes a budget-tuned router drift in cost and answer quality, or (b) how often cached answers are wrong because of earlier routing choices. **We do not claim** that combining a cache and a router is new; the table shows it isn't. What is new is *measuring what happens when you do*. The search behind this table was done on 21 September 2026 and is saved in our repository.

5. Research questions

- **RQ1 — Drift (minimum goal).** With a cache in front, how far do the router's share of strong-model calls, its cost, and its answer quality move from what was planned? We test several cache settings and several amounts of repeated questions.
- **RQ2 — Wrong cached answers (minimum goal).** How often is a cached answer wrong for the new question? We split this into false hits and inherited errors.
- **RQ3 — Re-tuning (minimum goal).** If we re-set α using only the questions the cache didn't answer, does the router return to its planned share and quality on new test questions?
- **RQ4 — Energy (stretch goal).** For one self-hosted model in two compressed versions, does using measured energy that changes with load, instead of one fixed number, change which model the router picks?

6. What we will build and how we will test it

6.1 The gateway (the product). A small server that an app can call the same way it calls the OpenAI API. Each question goes through three steps: **cache** → **router** → **model**. The cache uses embeddings and a similarity threshold. The router uses RouteLLM's published routers, plus a simple nearest-neighbor router as a second option. The gateway logs every decision, so we can report drift and wrong cached answers. It has two modes:

- **Live mode** calls real AI models. This is what a user would run.
- **Replay mode** takes each model's answer from a recorded dataset instead of a paid call. We use this for experiments, because it costs nothing and gives the same result every time.

Using the same gateway for both means our measurements come from the actual product, not from a separate simulation.

6.2 The test set. We start from RouterBench, which recorded answers from eleven models, including GPT-4 and Mixtral-8x7B-chat, with whether each answer was correct and what it cost [8, p. 5]. We pick questions from benchmarks with known correct answers (MMLU, GSM8K, ARC-Challenge and similar) and write several rewordings of each. **Correct answers always come from the original benchmarks, written by people. An AI may help reword questions, but it never decides what is correct.** A person checks a random sample of rewordings to make sure each still means the same thing. We then build streams of questions with an adjustable amount of repetition. We test a range of repetition rates rather than guessing one.

Replay mode relies on one assumption: a model that answers the original question correctly also answers its rewording correctly. We test that directly on a sample with live calls (assumption A3 in §10).

6.3 Experiments.

- **RQ1:** set α without the cache so that 20%, 50% or 80% of questions go to the strong model. Turn the cache on at five similarity settings and three repetition rates. Measure the real share, cost per 1,000 questions, and accuracy.
- **RQ2:** check every cached answer against the correct answer for the new question, and label each error as a false hit or an inherited error.
- **RQ3:** re-set α using only the cache misses from a tuning set, then test on a separate set. All rewordings of a question stay on the same side, so no test question has its twin in the tuning set.
- **RQ4 (stretch):** run one open model (Llama-3.1-8B) in full and 4-bit versions on a BU shared-cluster GPU. Read energy from NVIDIA's built-in counters (NVML) at several load levels, and check whether using these measured numbers changes the router's choices.

6.4 Measurement rules.

- Every number gets a 95% confidence interval.
- Anything random is repeated with five different random starting points (*seeds*).
- Cost, energy and accuracy are always reported in separate columns, never combined into one score.
- Only one setting changes per experiment.
- Tuning and test sets are fixed before we look at any results.

7. What "done" looks like after 12 weeks

The minimum successful project, in one sentence: a working gateway (cache → router → models) with a replay mode, plus a public report that answers RQ1–RQ3 with margins of error. The report covers how far the cache pushes the router off its budget, how often cached answers are wrong, and whether one-command re-tuning fixes it. All of it can be re-run with one command.

A "no" is still a result. If the drift turns out to be too small to matter at realistic settings, that is worth reporting: it tells engineers they can set their router once and stop worrying. RQ2 (wrong cached answers) stands on its own either way. Stretch goals, in order: (1) the energy measurements (RQ4); (2) testing the gateway under heavy traffic and when a model fails; (3) a vCache-style cache as a second cache design.

8. Milestones (five sprints)

Sprint	What we deliver
1	A bare-bones gateway running end to end in replay mode: one cache setting, one router, one budget. A one-day test of whether a cache shifts the router at all (A2). The rewording check (A3). Three user conversations, with a paper mock-up of the report (A1).
2	Test set version 1 (reworded groups and question streams). First drift and wrong-answer measurements, with margins of error.
3	All RQ1 and RQ2 settings; a separate test set; the first full results table.
4	RQ3 re-tuning built into the gateway as one command. Live-mode demo on a small real question stream. Stretch goal 1 (energy) if Sprint 3 finished on time.
5	Final report, public repository with automated tests, poster.

9. Project definition

9.1 Mission statement

For engineers who run an LLM feature under a monthly budget, with a cache in front of a model router, CacheShift is a gateway and measurement tool that shows how the cache shifts the router's cost and answer quality, and re-tunes the router for the questions it actually sees. Unlike gateway dashboards that only show how often the cache is used and how much it saves, it shows how good the cached answers are and how far the router has drifted from its budget, with margins of error.

Could someone prove this wrong? Yes, in three ways:

- the drift is too small to matter at realistic settings (RQ1);
- existing tools already report how good cached answers are;
- engineers don't actually tune routers to a budget (A1).

9.2 The Five W's

	CacheShift
What?	A gateway that saves money with a cache and a router, and reports whether those savings are hurting answer quality, with a one-command fix.
Who?	Uses it and decides: the engineer running the LLM feature. Also affected: the budget owner (spending) and the app's users (answer quality).
Why?	The router's budget was set without the cache. Turning the cache on silently changes both cost and quality, and today's dashboards only show savings, not whether answers are right [1].
When?	When the system is first set up, and whenever the cache setting or the traffic changes. For the course: a working gateway by Sprint 1, results by Sprint 3, the fix by Sprint 4.
Where?	Between the engineer's app and their AI models, as a drop-in server; plus a public repository with the report.
How?	Gateway (cache → router → models) with a replay mode → reworded test questions from RouterBench → drift, wrong-answer and re-tuning results with margins of error.

The line we can't fill in yet: **Who, by name.** See 9.3.

9.3 Users and what each gets

User	Role	What they get	How we'll know it works
Engineer running the LLM feature	Uses it and decides	The gateway, the drift report, and a one-command re-tune	They can tell in two minutes whether their budget still holds with the cache on
Budget owner	Pays for it	Cost per 1,000 questions with and without the cache, with a margin of error	The gap between planned and actual spending is explained
Router researcher	Secondary user	The public test set of reworded questions with correct answers	Can re-run any result with one command

The conflict this reveals: the engineer wants more savings (more cache hits), while the app's users want correct answers (fewer false hits). The report shows both, never one blended score.

Named user: no outside person yet. The closest real user is on our team. Spencer runs an AI service for a national K-12 assessment company, at about 1.3¢ per item, and faces exactly this setup question. The client isn't named because permission is still pending, and a teammate is a biased user. So three outside conversations (A1) are our first validation task: a BU team that runs an AI service, a maintainer of an open-source gateway (vLLM Semantic Router or LiteLLM), and one engineer in industry.

9.4 Top five user stories, biggest risk first

1. See the drift (can make or break the project). *As the engineer, I want to see how far my router's share of strong-model calls, its cost, and its accuracy change once the cache is on, so that I know whether my planned budget still holds.*

- Done when: for 3 budgets × 5 cache settings × 3 repetition rates, the report shows planned vs. real share, cost per 1,000 questions, and accuracy, each with a 95% confidence interval.
- Done when: a classmate outside the team can answer "does my budget still hold?" correctly from the report in under two minutes.

2. Know what a cached answer costs in quality. *As the engineer, I want every cached answer checked against the correct answer for the new question, so that I know whether my savings are coming out of answer quality.*

- Done when: false hits and inherited errors are reported separately for each cache setting, with confidence intervals.
- Done when: a person's check of 50 random cached answers agrees with the automatic labels at least 90% of the time.

3. Fix the drift. *As the engineer, I want one command that re-tunes the router on the questions that get past the cache, so that my real share returns to my plan.*

- Done when: on the separate test set, the real share of strong-model calls is within 3 percentage points of the plan, and accuracy is within the margin of error of the no-cache setup.

4. Use it on my app. *As the engineer, I want to point my app at the gateway instead of the AI provider, so that I get the savings and the drift report without changing my code.*

- Done when: an example app switches to the gateway by changing one web address.
- Done when: the gateway answers in live mode and writes a drift report from its own logs.

5. Trust the test set. *As a router researcher, I want reworded questions with correct answers written by people, so that the results don't depend on an AI grading itself.*

- Done when: correct answers come only from the benchmarks' answer keys.
- Done when: a person's check of 100 random rewordings finds at least 95% keep their meaning.
- Done when: the test set rebuilds with one command.

Rules every story must follow:

- Correct answers are never decided by an AI.
- One change per experiment.
- A margin of error on every number.
- Cost, energy and quality in separate columns.
- Tuning and test sets fixed in advance, with reworded groups kept together.
- Automated tests on every code change.
- Every claim limited to the models, versions and data we actually used.

10. Assumptions, tests, and when we would stop

Ranked by **how badly the project fails if the assumption is wrong** × **how unsure we are**. The Result column only reports what has actually been done.

#	Assumption	Fatal if wrong?	Cheapest test	Result
A1	Engineers really do put a cache in front of a budget-tuned router, and would use a drift report.	Yes. No user means no product.	Three 15-minute conversations, showing a paper mock-up.	Not run. Supporting evidence: gateway products ship both tools, and vLLM Semantic Router's work item asks for exactly this quality check [1].
A2	The drift is real, because the questions a cache answers are not a fair sample of the router's scores.	Yes for the main result; RQ2 still stands if it's wrong.	One-day test: score reworded Chatbot Arena questions [6] with a RouteLLM router [2], simulate the cache, and compare the questions it misses with all questions.	Not run. Sprint 1.
A3	A model that gets a question right also gets its rewording right, so recorded answers can stand in for new ones.	Medium. If wrong, experiments need paid live calls.	Live calls on about 200 rewordings for two models; measure how often results agree.	Not run.

#	Assumption	Fatal if wrong?	Cheapest test	Result
A4	Routers are tuned to a budget percentile, so drift is meaningful.	Yes. Otherwise RQ1 doesn't matter.	Read the tuning code.	Verified in RouteLLM: α is a percentile of scores on a fixed Chatbot Arena sample, and its documentation warns the real share "will differ based on the actual queries received" [3].
A5	Nobody has published this measurement.	Yes.	A search designed to prove us wrong.	Done 21 Sep 2026. Closest: Zhu et al. [4] (perfect cache), LLMBridge [16] (no interaction measured), RequestRouter [11] (no cache).
A6	We can read GPU energy on BU's shared cluster and run a 4-bit model there.	No. Only affects the stretch goal.	A five-minute energy reading and a short model test on one GPU.	Checked on paper. NVIDIA's documentation lists no special-access requirement for reading energy. vLLM's documentation lists 4-bit (AWQ) support on that GPU generation.

Why A1 comes before A2. If there's no drift, we still have a working gateway, a reusable test set, and the wrong-answer results. If there's no user, we have nothing anyone will adopt.

When we would stop, decided before testing. We change direction if, by the end of Sprint 2, **either** of these happens:

- **(a)** None of the three conversations turns up someone who would use a drift and cached-answer-quality report over what they have now.
- **(b)** Both the Sprint 1 test and the Sprint 2 results show drift and wrong-answer rates inside their margins of error at every realistic setting, **and** A3 fails, so the result is both empty and expensive to extend.

If only the "no drift" part happens, we keep going as a measurement study and say so in the title. If (a) happens, we change users or change projects.

Blind spots. The lecture asks us to have the LLM list the five assumptions most likely to sink the project. Its list, and our response to each:

1. **AI-written rewordings may be easier or harder than how real people reword questions.** We check them by hand and compare them with real Chatbot Arena rewordings.
2. **RouterBench's models are from 2023–24, so exact prices are out of date.** We report relative drift, not absolute cost.
3. **Real traffic may repeat far less than we test.** We include low repetition rates and say which rates are realistic, with sources.
4. **We are new to RouteLLM's code.** Sprint 1 runs its routers end to end inside the bare-bones gateway.
5. **The effect may be real but too small to matter in dollars.** We report drift as cost per 1,000 questions, so readers can judge.

11. Team

A two-person team: Spencer Aldrich and Anthony Capraru. We will share one repository, run automated tests on every code change, review each other's code, and rotate the sprint lead. How the work divides will be agreed at our first team meeting.

12. Risks

Risk	What we do
The drift is too small to matter (A2)	Report it as a measurement result. The gateway, the test set, and the wrong-answer results (RQ2) still stand.
Recorded answers don't carry over to rewordings (A3)	Set aside a small budget for live calls, and limit the experiments to the questions we've checked.
Someone publishes first (this area is active)	Cite all related work, keep our claim narrow, and publish results in our repository early.
AI-call costs run over	The main experiments use replay mode, so they cost nothing. Live calls are limited to small checks and the demo.
Two people is a small team	The minimum goal is sized for two people. Everything beyond it is a stretch goal.

13. References

All references were checked against the original papers (or, for [3], the source code) on 21 September 2026.

- [1] vLLM Semantic Router project, "[Epic] Make response caching safe, measurable, and lifecycle-aware," GitHub issue #3036, opened 26 Aug. 2026. (*Industry signal*.)
- [2] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "RouteLLM: Learning to Route LLMs with Preference Data," in *Proc. ICLR*, 2025. arXiv:2406.18665.
- [3] LMSYS, RouteLLM source code, `routellm/calibrate_threshold.py` and README, github.com/lmsys/RouteLLM (accessed 21 Sep. 2026).
- [4] B. Zhu, Y. Sheng, L. Zheng, C. Barrett, M. I. Jordan, and J. Jiao, "Towards Optimal Caching and Model Selection for Large Model Inference," in *Proc. NeurIPS*, 2023. arXiv:2306.02003 (arXiv title: "On Optimal Caching and Model Multiplexing for Large Model Inference").
- [5] F. Bang, "GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings," in *Proc. 3rd Workshop for NLP Open Source Software (NLP-OSS 2023)*, Singapore, 2023, pp. 212–218.
- [6] L. G. Schroeder et al., "vCache: Verified Semantic Prompt Caching," in *Proc. ICLR*, 2026. arXiv:2502.03771.
- [7] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance," *Transactions on Machine Learning Research*, Dec. 2024.
- [8] Q. J. Hu, J. Bieker, X. Li, N. Jiang, B. Keigwin, G. Ranganath, K. Keutzer, and S. K. Upadhyay, "RouterBench: A Benchmark for Multi-LLM Routing System," arXiv:2403.12031, 2024; ICML 2024 Agentic Markets Workshop. *Preprint / workshop*.
- [9] J. Lin et al., "AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration," in *Proc. MLSys*, 2024. arXiv:2306.00978.
- [10] T. Shi and Y. Ding, "Systematic Characterization of LLM Quantization: A Performance, Energy, and Quality Perspective," arXiv:2508.16712, 2025. *Preprint*.
- [11] A. Sunesh, A. Alshehhi, and H. Dhakne, "RequestRouter: Request-Boundary Routing for Efficient Single-GPU LLM Inference," in *2nd Int. Workshop on Low Carbon Computing (LOCO 2026)*, 2026. arXiv:2605.23057.

- [12] D. Sheshanarayana, R. S. Pal, M. Sinha, and T. Dasgupta, "GAR: Carbon-Aware Routing for LLM Inference via Constrained Optimization," arXiv:2605.11603, 2026. *Preprint*.
- [13] A. Bernhard and A. B. Yardimci, "Routing LLM Inference to the Cleanest Grid in Real Time," arXiv:2608.06188, 2026. *Preprint*.
- [14] U.S. National Science Foundation, "Design for Environmental Sustainability in Computing (DESC)," Program Solicitation NSF 23-532. (*Funding agenda*.)
- [15] Y Combinator, *Requests for Startups*, Fall 2026, item "Compute at Sea." (*Industry signal*.)
- [16] N. Martin, A. Bin Faisal, H. Eltigani, R. Haroon, S. Lamelas, and F. Dogar, "LLMBridge: Reducing Costs to Access LLMs in a Prompt-Centric Internet," arXiv:2410.11857, 2024. *Preprint*.